Discord OAuth2 Integration Plan

WordChain NFT Game · June 2026

Overview

This plan covers adding optional Discord identity to WordChain. After connecting a wallet, players can link their Discord account. Their display name and avatar then replace the wallet address on the leaderboard. The admin tool will also show whether Discord is connected for any given wallet. No OAuth tokens are persisted — only the minimum identity data (handle + avatar hash).

How It Works

Step 1 User connects wallet — Discord button becomes available.

Step 2 User clicks Connect Discord → OAuth2 popup opens.

Step 3 Discord redirects back with a one-time `code`.

Step 4 Backend exchanges code for access token, calls `GET /users/@me`, stores handle + avatar hash keyed to the wallet address.

Step 5 Access token is discarded. Leaderboard and admin tool reflect the linked identity immediately.

Scope required: `identify` *only — no email, no server membership, no bot needed.*

Data Stored (Minimal)

A single new table, keyed by wallet address:

```
address (PK) · discord_id · discord_handle · discord_avatar · connected_at
```

- **discord_id** — snowflake needed to construct the CDN avatar URL.
- **discord_handle** — `global_name` with fallback to `username`.
- **discord_avatar** — hash only; full URL built as `cdn.discordapp.com/avatars/{id}/{hash}.png`.
- No OAuth access or refresh tokens stored at any point.

Changes Required

1. Discord App Setup

- Register an application at discord.com/developers.
- Add your OAuth2 redirect URI (e.g. yourdomain.com/discord-callback).
- Copy CLIENT_ID and CLIENT_SECRET into environment variables.

2. Database Migration

- Add the discord_connections table (schema above).
- Index on address for O(1) leaderboard joins.

3. New API Endpoint — POST /api/auth/discord

- Accepts { code, wallet_address } + valid wallet JWT.
- Exchanges code, fetches /users/@me, upserts discord_connections.
- Also expose DELETE /api/auth/discord to unlink.
- Access token never leaves this function.

4. Leaderboard Handler Update

- JOIN discord_connections in the existing leaderboard query.
- Return discord_handle and discord_avatar alongside address.
- Frontend shows name/avatar when present, truncated address otherwise.

5. Admin Handler Update

- lookup-player action gains a discord field in its response:

```
{ "connected": true, "handle": "Isuma", "avatar_url": "..." }
```

6. Frontend Flow

- Show Connect Discord button once wallet auth succeeds.
- On click: open Discord OAuth2 authorize URL in a popup.
- Handle callback, call POST /api/auth/discord, refresh UI.
- Leaderboard rows with Discord linked show avatar + handle.

Effort Estimate

Task	Est. Time
Discord app setup	30 min
DB migration	30 min
Backend OAuth endpoint	3 hrs
Leaderboard + admin updates	2 hrs
Frontend flow + UI	3–4 hrs
Total	**~9–10 hrs**

Roughly 1–2 days of focused development.

Feasibility & Caveats

- **global_name can be null** for legacy accounts still on the username#discriminator system. Always fall back to username.
- **Avatar URLs** are constructed at render time from the stored hash — no need to cache full URLs. Default Discord avatars apply when avatar hash is null.
- **Access tokens expire in 7 days** — since you discard them immediately after the one-time fetch, this is not a concern. If you ever want periodic name/avatar refresh, store a refresh token at that point.
- **No rate-limit risk** — /users/@me is called once per user link action only.
- **Overall: 100% feasible** with the identify scope. No bot, no server membership, no sensitive data required.

Recommended starting point: DB migration + backend OAuth endpoint.